Final Project
Development of Large Systems

Project description

The goal of this project is to develop a system with a distributed architecture suitable for large IT systems. We are aiming for characteristics like:

- Scalability:
  - databases: amount of data, number of requests, query optimization.
  - application: microservices architecture enabling the use of Kubernetes to orchestrate the containerized microservices.
  - Proper API design enabling easy extensions and modifications.
- Portability: containerization – each microservice is shipped with all the dependencies and runtime environments.
- Security: Authentication and authorization, …
- Interoperability: designing proper APIs
- AI Integration: The system must include integration with an AI-based service. The AI functionality must be accessed via a well-defined API and integrated into at least one backend microservice as part of a real system workflow.

There must be a minimum of 5 microservices – this means 5 separate backend applications. API Gateway does not count unless it contains business logic.

The project must also include a frontend client (does not count as a microservice).

Important notes: Services like databases, message brokers, frontend clients, 3rd-party services, etc. do not count as microservices.

For this project you are free to choose any development stack – programming languages, frameworks, cloud providers, database servers, etc.

You must write the specifications for your project including the functional and non-functional requirements.

From the very beginning of the project, you should set up a proper project management strategy (for the team collaboration) and use a version control system like git/GitHub and set up a CI/CD pipelines (for example using GitHub Actions) with automated tests. Part of the delivery will be a link to the monorepo hosting all the microservices where contributions from each student will be clearly visible via git history.

Environments:

- Development environment should be configured using docker-compose.
- Production environment should be simulated using local Kubernetes cluster. It is not required to make a full cloud deployment, but it should be described in the report – especially how it will support properties like elasticity, high availability, and low latency.

Examples of suitable business ideas for the project:

- Stock brokerage web app
- Logistics & Package Tracking System
- Hospital & Patient Record System
- Hotel booking & room allocation system
- Payroll & employee management system
- Restaurant ordering & kitchen workflow system
- Warehouse & inventory optimization system
- Ride-sharing & trip tracking system
- Social media content & moderation backend
- Real-estate listing & rental system
- Food delivery & courier dispatch system
- Manufacturing production tracking system
- Banking system
- Movie rental / streaming system
- Car rental system
- Web shop
- Airline reservation system
- …

The project must be complex enough to cover the curriculum of the whole course. Students are responsible for selecting a suitable project.

Architecture, Communication, and Quality Requirements

The following requirements must be implemented in the project.
The course will cover the theoretical background and motivation for each requirement.

1. Communication and Integration

- Backend microservices must primarily communicate with each other **asynchronously** using **message queues**.
- The backend must expose APIs to the frontend using **both REST and GraphQL**:
  - At least one microservice must expose a REST API.
  - At least one microservice must expose a GraphQL API.
    (Both approaches may be implemented in the same microservice or in separate microservices.)
- The system must demonstrate **API interoperability** between frontend and backend services, so that different API styles exposed by the backend work together seamlessly when consumed by the frontend as part of real system workflows.
- The frontend application must consume backend APIs using both REST and GraphQL and interact with multiple backend microservices.
- The project must include **authentication and authorization**, including:
  - Token-based authentication (e.g. JWT).
  - Integration with at least one third-party identity provider using OAuth/OpenID Connect (e.g. Google, Microsoft, GitHub, Facebook).
- The system must integrate with an AI-based service via an API, used by at least one backend microservice as part of its business logic.

2. System Architecture and Design Patterns

- Each backend microservice must be independently deployable.
- Each microservice must own its data and must not directly access another microservice's database.
- The project must demonstrate the use of **advanced distributed systems patterns**, including:
  - **CQRS (Command Query Responsibility Segregation)** where appropriate – the implementation needs to be justified.
  - **Immutable data handling**, using:
    - Tombstone pattern and/or
    - Snapshot pattern.
  - **Idempotent operations**, ensuring that repeated client requests do not cause duplicate side effects.
  - **Commutative message handlers**, where message order does not affect the final system state.
  - **Saga pattern** to ensure consistency across multiple services or databases using compensating actions rather than distributed transactions.

3. Cloud-Native and Operational Concerns

- The system must include a **logging and monitoring solution** suitable for distributed systems.
- The project must demonstrate **cloud-native design principles**, including:
  - Containerized services.
  - Clear deployment architecture.
  - The CI/CD pipeline must include automated execution of unit, integration, and system-level cooperation tests.
- The AI service may be simulated locally using a containerized local LLM (for example using tools such as Ollama), in order to avoid dependency on paid external APIs.
- **Serverless functions** must be used for at least one isolated task (e.g. image processing, background jobs, notifications). It can be simulated in local Kubernetes deployment (for example, a containerized console application scaled using KEDA as a ScaledJob).

4. Versioning Strategy

A consistent versioning strategy must be implemented and documented:

- **Source code versioning** using Git and GitHub or other platforms.
- **Database versioning** using database migration tools.
- **API versioning** for REST APIs
  (GraphQL does not require versioning by design.)

5. Testing Requirements

- The project must include automated testing demonstrating correct behavior of individual services and their cooperation within the distributed system.
- The project must include basic security testing validating authentication and authorization behavior.
- Static analysis or static testing tools must be used to automatically detect code quality or security issues and must be executed as part of the CI/CD pipeline.
- Each backend microservice must include automated unit tests covering core business logic.
- The project must include automated integration tests validating at least one of each of the following:
  - REST or GraphQL API behavior
  - Asynchronous communication via message queues
  - Database integration
- The project must include at least one system-level cooperation test demonstrating correct interaction between two or more backend services.
  - The test must start one or more dependent services (for example using containers).
  - The test must verify that the services cooperate as intended (typically via messaging).
  - The test must focus on service interaction, not on UI or end-to-end user workflows.
- All tests must be executed automatically as part of the CI/CD pipeline.
- Test failures must cause the CI/CD pipeline to fail.

6. Documentation Requirements

Comprehensive documentation is mandatory and must include:

- GitHub **README** for the whole monorepo and for each microservice, and, if relevant, GitHub Wiki.
- **Swagger / OpenAPI documentation** for all REST APIs.
- **GraphQL schema and endpoint documentation**.
- **AsyncAPI documentation** (or an equivalent standard) for all message-based communication, including:
  - Message brokers, queues, or topics
  - Message schemas and payload formats
  - Producers and consumers
  - Event flows between microservices
- Overall **system documentation**, including:
  - Architecture diagrams
  - Description of communication between services
  - Explanation of applied patterns and design decisions

Final Project Delivery to WISEflow

Final Project Artifacts (things to deliver)

- Report
- Links to a GitHub monorepo containing all the source code
  - Documentation:
    - GitHub readmes, wiki
    - swagger for REST APIs
- A brief installation procedure that describes how to organize the codebase and initialize the databases in a development environment with full operational capabilities, for example to support onboarding a new group member and setting up a local development and Kubernetes environment.

Final Project Report

Official requirements for the report can be found in the course catalog:
https://katalog.kea.dk/course/9942251/2025-2026

The report should have this structure:

Cover page, including Title, Full names of all students in the group, Date of delivery
List of figures
List of appendices
Table of contents (paginated index)

1. **Introduction**
   1.1. Problem description

1.2. Functional and non-functional requirements
1.3. Explanation of choices for the technology stack (databases, programming languages, frameworks, etc.).

2. **System architecture**
   2.1. Introduction to the microservices architecture + architectural diagram of the whole system
   2.2. Microservices Description
   For each backend microservice, the following must be explained. Diagrams and API specifications should be referenced rather than duplicated. Implementation details and code-level explanations should be avoided.
      - Purpose and responsibilities, including why the functionality was isolated into this microservice
      - Exposed APIs (REST / GraphQL / asynchronous), including rationale for the chosen communication style
      - Consumed and produced events, and the role of this microservice in asynchronous workflows
      - Data storage and ownership, including justification for database choice and schema boundaries
      - Key business logic, focusing on behavior rather than implementation details
      - Scalability and failure considerations, including how the service behaves under load or failure

   2.3. Communication between microservices
   2.4. Description of the patterns and techniques used in the project.
   Only patterns that are implemented must be described in detail; other relevant patterns may be discussed conceptually but only if justified.
      2.4.1. CQRS
      2.4.2. Immutable data (Tombstone pattern and Snapshot pattern)
      2.4.3. Idempotence
      2.4.4. Commutative message handlers
      2.4.5. Saga pattern
      2.4.6. Caching
      2.4.7. …

3. **Environments description**
   3.1. Development environment with docker-compose including the architecture diagram
   3.2. Local Kubernetes deployment simulating a production-like environment including the architecture diagram:
      3.2.1. Description of used technologies
      3.2.2. CI/CD pipeline description and diagram
      3.2.3. Monitoring and logging of the deployed system
      3.2.4. Autoscaling configuration
   3.3. Cloud deployment plan – deployment diagram, selected cloud providers and services, pricing strategies, scalability and availability considerations (both app and database

level), security and data management strategies, deployment and release approaches, and justified architectural trade-offs. Actual cloud deployment is not required.

4. **Testing**
   - 4.1. Testing strategy and scope
   - 4.2. Unit testing
   - 4.3. Integration testing
   - 4.4. System-level cooperation testing
   - 4.5. Security testing focusing on verifying correct authentication and authorization
   - 4.6. Static analysis and static testing to identify code quality or security issues.
   - 4.7. Test execution in CI/CD
   - 4.8. Limitations, risks, and trade-offs
5. **Project management and team collaboration**
   - 5.1. Introduction to the project management and team collaboration
   - 5.2. Description of the methods used during the project.
   - 5.3. Versioning strategies for the source code, databases, and APIs
   - 5.4. Documentation strategy
6. **Discussion**
   - 6.1. Advantages and challenges of the distributed systems (application and database perspectives)
   - 6.2. Pros and cons of used patterns like CQRS etc.
   - 6.3. Scalability / Autoscaling (application, messaging, and database perspectives)
   - 6.4. Possible improvements
7. **Reflection:** What was difficult about: designing the distributed system, project management, what design mistakes were made and fixed, what would be done differently next time, what you learned about databases in practice.
8. **Conclusion:** short high-level summary of the project, results, main insights.
9. **References:** All major design choices, technical claims, and comparisons must be supported by references. Sources must be cited in the text and listed in the References section. References should primarily be official documentation, textbooks, or academic or technical publications. Web tutorials and blogs may be used sparingly but should not be the primary sources. Use standard formats like APA, IEEE, or Harvard.
10. **Appendix**